SatsPath P2P Wire Transport v1 (Deprecated/Archived)

> [!WARNING] **This transport has been deprecated and retired from the canonical SatsPath architecture.** SatsPath has fully transitioned to authoritative S2S resolution (v2). P2P discovery is no longer required or supported by default. This document remains solely for historical reference.

Pear/Holepunch is an optional transport for SatsPath protocol objects. It is not the SatsPath protocol.

This document defines how a P2P transport carries signed profiles and resolver messages while preserving the protocol rules in protocol.md.

Transport Role

P2P transport can be used to:

- Announce that a peer has a signed payment profile for an identifier.
- Request a signed profile for an identifier.
- Return a `SignedPaymentProfile`.
- Exchange resolver diagnostics.

P2P transport MUST NOT:

- Replace profile signatures.
- Carry wallet seeds or spending keys.
- Authorize payments by itself.
- Treat peer connectivity as proof of profile ownership.

Topic Derivation

The reference Holepunch SDK uses a topic derived from a canonicalized alias:

```
topic = SHA256("satspath:p2p:v1:" + canonical_identifier)
```

The topic hides the raw alias from casual discovery but is not a privacy guarantee against dictionary attacks. Clients SHOULD avoid announcing sensitive identifiers unless that tradeoff is acceptable.

Message Envelope

All P2P messages SHOULD use a versioned envelope:

```
{
  "protocol": "satspath",
  "version": 1,
  "type": "profile_response",
  "request_id": "01J...",
  "body": {}
}
```

Fields:

- `protocol`: MUST be `satspath`.
- `version`: MUST be `1` for this wire version.
- `type`: Message type.
- `request_id`: Optional caller-generated correlation ID.
- `body`: Type-specific payload.

Implementations MAY carry raw `SignedPaymentProfile` objects for compatibility, but versioned envelopes are preferred for forward compatibility.

Message Types

`profile_announce`

Announces that a peer can serve a signed profile.

```
{
  "protocol": "satspath",
  "version": 1,
  "type": "profile_announce",
  "body": {
    "alias_hash": "be8979...",
    "identity_fingerprint": "a8fdac91",
    "updated_at": 1782810000
  }
}
```

`alias_hash` is a lookup hint, not proof of ownership.

`profile_request`

Requests a signed profile.

```
{
  "protocol": "satspath",
  "version": 1,
  "type": "profile_request",
  "request_id": "req_123",
  "body": {
    "identifier": "alice@example.com"
  }
}
```

Peers MAY reject requests or rate-limit responses.

`profile_response`

Returns a signed profile.

```
{
  "protocol": "satspath",
  "version": 1,
  "type": "profile_response",
  "request_id": "req_123",
  "body": {
    "signed_profile": {
      "profile": {
        "alias": "alice@example.com",
        "identity_pubkey": "02...",
        "methods": []
      },
      "signature": "3044..."
    }
  }
}
```

Receivers MUST verify `signed_profile` before importing, displaying as verified, or routing.

`profile_not_found`

Indicates the peer does not have the requested profile.

```
{
  "protocol": "satspath",
  "version": 1,
  "type": "profile_not_found",
  "request_id": "req_123",
  "body": {
    "identifier_hash": "be8979..."
  }
}
```

`error`

Returns a transport-level error.

```
{
  "protocol": "satspath",
  "version": 1,
  "type": "error",
  "request_id": "req_123",
  "body": {
    "code": "rate_limited",
    "message": "Too many profile requests"
  }
}
```

Verification Rules

On receipt of a P2P profile response, a client MUST:

1. Decode JSON.
2. Confirm `protocol == "satspath"` and `version == 1` if using an envelope.
3. Decode `SignedPaymentProfile`.
4. Verify the profile signature.
5. Check expiry.
6. Reject private material.
7. Optionally persist the verified profile into the local registry.

Peer identity, DHT topic, or transport connection state MUST NOT bypass these checks.

Connection State

Implementations SHOULD expose connection diagnostics separately from protocol verification:

```
{
  "kind": "p2p_bridge",
  "status": "started",
  "active": true,
  "detail": "pid:1234"
}
```

Connection state can explain discovery failures, but it does not prove payment safety.

Reference Implementation

The reference P2P transport lives in:

`sdk/satspath-p2p/`

The daemon can start an optional bridge:

```
satspathd --p2p
```

The bridge publishes the local signed profile over the P2P transport. The same profile can also be served over HTTPS, imported from a file, or discovered through other resolvers.

Compatibility

Version 1 receivers SHOULD ignore unknown envelope fields. Version 1 receivers MUST reject unknown required message semantics if a future message declares them.

Future wire versions may change message types or add encryption, but must preserve the core protocol invariant: profiles are verified as SatsPath signed profiles before routing.